

BUS RESERVATION & FLEET SCHEDULING SYSTEM

DBMS Course Project Report

Student Name	M.samyuktha
Roll Number	25WU0102241
Course	Database Management Systems (DBMS)
Academic Year	2025-2029
University	Woxsen University
Branch	CSE - AIML
Project Type	DBMS Course Project

A relational database-driven web application for managing routes, buses, schedules, seats, passengers, bookings, crew, payments and cancellations.

2. Abstract

The Bus Reservation & Fleet Scheduling System is a database-driven web application designed to manage the complete reservation workflow in a structured relational database. The system maintains route and stop information, bus and seat inventories, schedules, crew assignments, passenger records, bookings, payments and cancellations. A connected web interface provides operational views for booking, passenger management, scheduling and DBMS reporting. MySQL acts as the persistent relational data layer, while the application layer connects the interface to the database and exposes the required operations.

The design emphasizes referential integrity, controlled booking status, unique seat allocation per trip, payment tracking and cancellation/refund records. The system also supports SQL-based reporting through joins, aggregation and related queries, making it suitable for demonstrating core DBMS concepts in an industry-style operational interface.

3. Introduction and Problem Statement

3.1 Introduction

Bus reservation operations generate strongly related records: routes contain stops, buses contain seats, schedules operate a route using a bus, employees are assigned to trips, passengers create bookings, bookings generate payments and selected bookings may be cancelled. Managing these records independently can cause duplicate information, incorrect seat availability and inconsistent financial status. The proposed system models the workflow as a normalized relational database connected to a web interface.

3.2 Problem Statement

The system requires a reliable way to maintain route and bus data, publish schedules, reserve seats, register passengers, assign crew, record payments and process cancellations. It must preserve relationships between these records and prevent invalid operations such as assigning unavailable seats, referencing non-existent passengers or trips, recording payments without bookings, or creating duplicate cancellation records.

4. Objectives and Scope

4.1 Objectives

- Maintain routes, stops, buses, seats, employees, schedules, crew assignments, passengers, bookings, payments and cancellations in MySQL.
- Provide a booking workflow that searches a route and date, selects a schedule, allocates a seat, captures passenger details and confirms the reservation.
- Maintain one payment record for each booking and track paid/refunded status.
- Record cancellations with reasons and refund amounts.
- Support operational reports using joins, aggregation, grouping and derived calculations.
- Provide a clean web interface for database-backed operational tasks.

4.2 Scope

- Route and stop management.
- Bus and seat inventory management.
- Trip scheduling and crew assignment.
- Passenger registry and booking management.
- Payment and cancellation management.
- Operational reporting and dashboard views.
- Local web application deployment connected to MySQL.

5. Software and Hardware Requirements

Category	Requirement
Database	MySQL 8.0
Frontend	HTML5 / CSS / JavaScript web interface
Backend	Application backend/API connected to MySQL
DB Connectivity	MySQL connector/driver
Browser	Modern Chromium / Firefox / Safari
Deployment	Localhost / local development environment
Recommended Hardware	Dual-core processor, 8 GB RAM, 10 GB free storage

6. ER Diagram and Relationship Model

The system is organized around eleven relational entities. Routes and Stops define travel corridors; Buses and Seats define vehicle capacity; Schedules connect routes and buses for a particular trip; Crew_Assignment connects employees to trips; Passengers connect to Bookings; Bookings reference a trip and seat; Payments and Cancellations extend the booking lifecycle.

ENTITY	PRIMARY LINK	RELATIONSHIP
Routes	route_id	Routes → Stops; Routes → Schedules
Stops	stop_id	Each stop belongs to one route
Buses	bus_id	Buses → Seats; Buses → Schedules
Seats	seat_id	Each seat belongs to one bus; Bookings select a seat
Employees	emp_id	Employees → Crew_Assignment
Schedules	trip_id	Schedule links one route + one bus; used by Bookings
Crew_Assignment	assignment_id	Trip ↔ Employee with Driver/Conductor role
Passengers	passenger_id	Passenger → Bookings
Bookings	booking_id	Central reservation record; → Payments / Cancellations
Payments	payment_id	One payment per booking
Cancellations	cancellation_id	Optional cancellation per booking

Core flow: Route → Schedule → Bus → Seat → Booking ← Passenger → Payment / Cancellation

7. Relational Schema and Normalization

7.1 Relational Schema

The following tables represent the implemented MySQL structure used by the project.

7.1 Routes

Field	Type	Null	Key	Default / Extra
route_id	INT	NO	PRI	AUTO_INCREMENT
source	VARCHAR(100)	NO		
destination	VARCHAR(100)	NO		
distance_km	DECIMAL(8,2)	NO		

7.1 Stops

Field	Type	Null	Key	Default / Extra
stop_id	INT	NO	PRI	AUTO_INCREMENT
route_id	INT	NO	MUL	
stop_name	VARCHAR(100)	NO		
stop_order	INT	NO		

7.3 Buses

Field	Type	Null	Key	Default / Extra
bus_id	INT	NO	PRI	AUTO_INCREMENT
bus_number	VARCHAR(20)	NO	UNI	
bus_type	VARCHAR(50)	NO		
total_seats	INT	NO		

7.3 Seats

Field	Type	Null	Key	Default / Extra
seat_id	INT	NO	PRI	AUTO_INCREMENT
bus_id	INT	NO	MUL	
seat_number	VARCHAR(10)	NO		
seat_type	VARCHAR(30)	YES		Regular

7.5 Employees

Field	Type	Null	Key	Default / Extra
emp_id	INT	NO	PRI	AUTO_INCREMENT
emp_name	VARCHAR(100)	NO		
phone	VARCHAR(15)	NO	UNI	
role	VARCHAR(20)	NO		
salary	DECIMAL(10,2)	YES		

7.5 Schedules

Field	Type	Null	Key	Default / Extra
trip_id	INT	NO	PRI	AUTO_INCREMENT
route_id	INT	NO	MUL	
bus_id	INT	NO	MUL	
trip_date	DATE	NO		
departure_time	DATETIME	NO		
arrival_time	DATETIME	NO		
base_fare	DECIMAL(8,2)	NO		
status	VARCHAR(15)	NO		Scheduled

7.7 Crew_Assignment

Field	Type	Null	Key	Default / Extra
assignment_id	INT	NO	PRI	AUTO_INCREMENT
trip_id	INT	NO	MUL	
emp_id	INT	NO	MUL	
duty_role	VARCHAR(20)	NO		

7.7 Passengers

Field	Type	Null	Key	Default / Extra
passenger_id	INT	NO	PRI	AUTO_INCREMENT
first_name	VARCHAR(50)	NO		
last_name	VARCHAR(50)	NO		
gender	VARCHAR(10)	YES		
date_of_birth	DATE	YES		
phone	VARCHAR(15)	NO	UNI	
email	VARCHAR(100)	YES	UNI	
address	VARCHAR(255)	YES		

7.9 Bookings

Field	Type	Null	Key	Default / Extra
booking_id	INT	NO	PRI	AUTO_INCREMENT
trip_id	INT	NO	MUL	
seat_id	INT	NO	MUL	
passenger_id	INT	NO	MUL	
booking_date	DATETIME	YES		CURRENT_TIMESTAMP
fare_amount	DECIMAL(8,2)	NO		
status	VARCHAR(15)	NO		Confirmed

7.9 Payments

Field	Type	Null	Key	Default / Extra
payment_id	INT	NO	PRI	AUTO_INCREMENT
booking_id	INT	NO	UNI	
payment_date	DATETIME	YES		CURRENT_TIMESTAMP
amount	DECIMAL(8,2)	NO		
payment_method	VARCHAR(20)	NO		
payment_status	VARCHAR(20)	NO		Paid

7.11 Cancellations

Field	Type	Null	Key	Default / Extra
cancellation_id	INT	NO	PRI	AUTO_INCREMENT
booking_id	INT	NO	UNI	
cancellation_date	DATETIME	YES		CURRENT_TIMESTAMP
reason	VARCHAR(255)	YES		
refund_amount	DECIMAL(8,2)	YES		0

7.2 Normalization

The schema follows 1NF, 2NF and 3NF principles. Each table represents a focused entity, repeating groups are avoided, and relationships are represented through foreign keys. Crew_Assignment resolves the relationship between trips and employees, while Bookings links passengers, trips and seats without duplicating route or bus master data. Payments and Cancellations are separated from Bookings to preserve clear lifecycle records.

8. Database Creation and Sample Data

8.1 Database Initialization

The database is initialized as bus_reservation_db and populated with master and transactional data.

Seeded data used for the application demonstration:

Table	Records / Capacity
Routes	5
Stops	15
Buses	5
Seats	200 (40 per bus)
Employees	10
Schedules	10
Crew_Assignment	20
Passengers	20
Bookings	20
Payments	20
Cancellations	5

8.2 Representative SQL Operations

The implementation uses INSERT operations for master and transactional data, JOIN queries for operational views, aggregation for reporting, and foreign keys to maintain referential integrity.

```
SELECT
  b.booking_id,
  CONCAT(p.first_name, ' ', p.last_name) AS passenger,
  CONCAT(r.source, ' → ', r.destination) AS route,
  s.trip_date,
  bus.bus_number,
  seat.seat_number,
  b.fare_amount,
  b.status AS booking_status,
  pay.payment_status,
  c.cancellation_id
FROM Bookings b
JOIN Passengers p ON b.passenger_id = p.passenger_id
JOIN Schedules s ON b.trip_id = s.trip_id
JOIN Routes r ON s.route_id = r.route_id
JOIN Buses bus ON s.bus_id = bus.bus_id
JOIN Seats seat ON b.seat_id = seat.seat_id
LEFT JOIN Payments pay ON b.booking_id = pay.booking_id
LEFT JOIN Cancellations c ON b.booking_id = c.booking_id
ORDER BY b.booking_id;
```

9. Queries with Outputs / Operational Reporting

9.1 Revenue by Route

Revenue by route is calculated from confirmed bookings by joining Routes, Schedules and Bookings and aggregating fare amounts.

SQL Query

```
SELECT
    CONCAT(r.source, ' ? ', r.destination) AS route,
    SUM(b.fare_amount) AS total_revenue
FROM Routes r
JOIN Schedules s
    ON r.route_id = s.route_id
JOIN Bookings b
    ON s.trip_id = b.trip_id
WHERE b.status = 'Confirmed'
GROUP BY r.route_id, r.source, r.destination
ORDER BY total_revenue DESC;
```

Output

route	total_revenue
Hyderabad ? Bengaluru	3600.00
Hyderabad ? Chennai	2000.00
Vijayawada ? Chennai	1950.00
Hyderabad ? Warangal	1200.00
Hyderabad ? Vijayawada	900.00

5 rows in set (0.00 sec)

Figure 9.1. Revenue by route query output.

9.2 Booking Statistics

SQL Query

```
SELECT
    status,
    COUNT(*) AS total_bookings
FROM Bookings
GROUP BY status
ORDER BY total_bookings DESC;

-- Confirmed / Cancelled / Pending
SELECT
    SUM(CASE WHEN status = 'Confirmed' THEN 1 ELSE 0 END) AS confirmed,
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled,
    0 AS pending
FROM Bookings;
```

Output 1 · Booking Status

```
-> ORDER BY total_bookings DESC;
+-----+-----+
| status | total_bookings |
+-----+-----+
| Confirmed | 15 |
| Cancelled | 5 |
+-----+-----+
2 rows in set (0.00 sec)
```

Output 2 · Confirmed / Cancelled / Pending

```
-> FROM Bookings;
+-----+-----+-----+
| confirmed | cancelled | pending |
+-----+-----+-----+
| 15 | 5 | 0 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Figure 9.2. Booking statistics query outputs.

9.3 Trip Occupancy

Trip occupancy is calculated from confirmed bookings against each bus capacity and expressed as a percentage.

SQL Query

```
SELECT
    s.trip_id,
    CONCAT(r.source, ' ? ', r.destination) AS route,
    s.trip_date,
    COUNT(b.booking_id) AS booked_seats,
    bus.total_seats AS capacity,
    ROUND((COUNT(b.booking_id) / bus.total_seats) * 100, 2)
        AS occupancy_percentage,
    s.status
FROM Schedules s
JOIN Routes r ON s.route_id = r.route_id
JOIN Buses bus ON s.bus_id = bus.bus_id
LEFT JOIN Bookings b
    ON s.trip_id = b.trip_id
    AND b.status = 'Confirmed'
GROUP BY s.trip_id, r.source, r.destination,
    s.trip_date, bus.total_seats, s.status
ORDER BY s.trip_date;
```

Output

```
> ORDER BY s.trip_date;
```

trip_id	route	trip_date	booked_seats	capacity	occupancy_percentage	status
1	Hyderabad ? Vijayawada	2026-10-10	1	40	2.50	Scheduled
2	Hyderabad ? Bengaluru	2026-10-10	2	40	5.00	Scheduled
3	Hyderabad ? Chennai	2026-10-11	1	40	2.50	Scheduled
4	Vijayawada ? Chennai	2026-10-11	1	40	2.50	Scheduled
6	Hyderabad ? Vijayawada	2026-10-12	1	40	2.50	Scheduled
5	Hyderabad ? Warangal	2026-10-12	2	40	5.00	Scheduled
7	Hyderabad ? Bengaluru	2026-10-13	2	40	5.00	Scheduled
8	Hyderabad ? Chennai	2026-10-14	1	40	2.50	Scheduled
9	Vijayawada ? Chennai	2026-10-15	2	40	5.00	Scheduled
10	Hyderabad ? Warangal	2026-10-16	2	40	5.00	Scheduled

10 rows in set (0.00 sec)

Figure 9.3. Trip occupancy query output.

9.4 Passenger Booking Count

SQL Query

```
SELECT
  p.passenger_id,
  CONCAT(p.first_name, ' ', p.last_name) AS passenger_name,
  COUNT(b.booking_id) AS total_bookings
FROM Passengers p
LEFT JOIN Bookings b
  ON p.passenger_id = b.passenger_id
GROUP BY p.passenger_id, p.first_name, p.last_name
ORDER BY total_bookings DESC;
```

Output

passenger_id	passenger_name	total_bookings
1	Rahul Sharma	1
2	Priya Reddy	1
3	Arjun Kumar	1
4	Sneha Patel	1
5	Vikram Rao	1
6	Ananya Iyer	1
7	Karthik Reddy	1
8	Meghana Sharma	1
9	Rohan Verma	1
10	Aishwarya Nair	1
11	Aditya Singh	1
12	Pooja Krishna	1
13	Sanjay Reddy	1
14	Divya Menon	1
15	Manish Gupta	1
16	Kavya Rao	1
17	Nikhil Reddy	1
18	Swathi Naidu	1
19	Varun Chandra	1
20	Harini Das	1

20 rows in set (0.00 sec)

Figure 9.4. Passenger booking count query output.

9.5 Cancellation / Refund Analysis

SQL Query

```
SELECT
  c.cancellation_id,
  b.booking_id,
  CONCAT(p.first_name, ' ', p.last_name) AS passenger_name,
  CONCAT(r.source, ' ? ', r.destination) AS route,
  c.reason,
  c.refund_amount,
  pay.payment_status
FROM Cancellations c
JOIN Bookings b ON c.booking_id = b.booking_id
JOIN Passengers p ON b.passenger_id = p.passenger_id
JOIN Schedules s ON b.trip_id = s.trip_id
JOIN Routes r ON s.route_id = r.route_id
LEFT JOIN Payments pay ON b.booking_id = pay.booking_id
ORDER BY c.cancellation_date DESC;
```

Output

cancellation_id	booking_id	passenger_name	route	reason	refund_amount	payment_status
1	2	Priya Reddy	Hyderabad ? Vijayawada	Personal reasons	450.00	Refunded
2	5	Vikram Rao	Hyderabad ? Chennai	Change of travel plans	1000.00	Refunded
3	8	Meghana Sharma	Vijayawada ? Chennai	Emergency situation	650.00	Refunded
4	11	Aditya Singh	Hyderabad ? Vijayawada	Travel date changed	450.00	Refunded
5	15	Manish Gupta	Hyderabad ? Chennai	Personal reasons	1000.00	Refunded

5 rows in set (0.00 sec)

Figure 9.5. Cancellation and refund analysis query output.

10. UI Design and Screenshots

The implemented interface follows an operational dashboard style with a restrained green palette, clear navigation and database-driven screens. The following screenshots document the working frontend.

OPERATIONS
Bus Reservation System

Dashboard
Bookings
Passengers
Schedules
Routes & Stops
Buses & Seats
Crew
Payments
Cancellations
Reports

SYSTEM STATUS
Database Connected

Search routes, bookings, passengers

Tue, 06 Oct, 2026 ■ Live Ops Admin

BOOKINGS
Issue a reservation
Search a trip, lock a unique seat, capture passenger details, then confirm. One booking per seat per trip is enforced.

Step 1 Search trip Step 2 Select seat Step 3 Passenger Step 4 Confirm

SOURCE DESTINATION TRAVEL DATE

Hyderabad Bengaluru 06-10-2026 Find schedules

Search records

ROUTE	BUS	DEPARTURE	ARRIVAL	BASE FARE	AVAILABLE	STATUS
Search a corridor to list available Schedules.						

0 records < 1/1 >

Recent bookings

Figure 1. Booking workflow — schedule search, seat selection, passenger capture and confirmation.

Bus Reservation System

Dashboard

Bookings

Passengers

Schedules

Routes & Stops

Buses & Seats

Crew

Payments

Cancellations

Reports

SYSTEM STATUS

Database Connected

Search routes, bookings, passengers

Tue, 06 Oct, 2026LiveOps Admin

PASSENGERS

Passenger registry

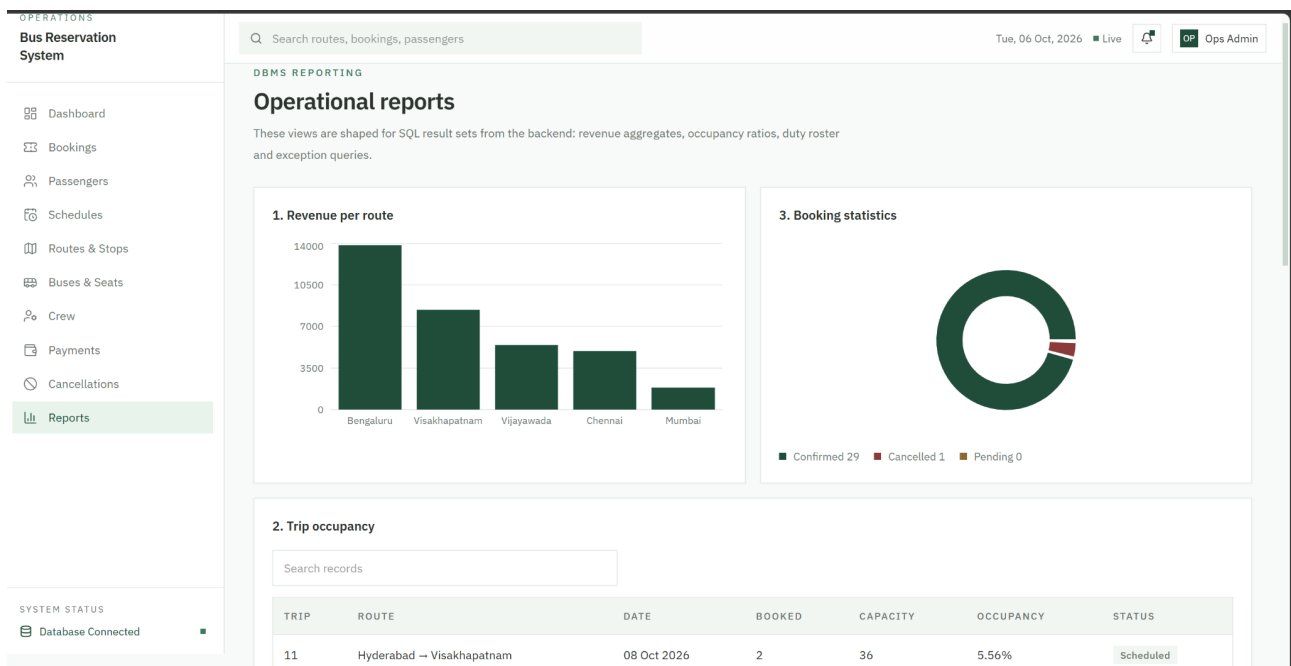
Master data from the Passengers table, with booking counts derived from Bookings.

Add passenger

Search passengers

PASSENGER ID	NAME	PHONE	EMAIL	GENDER	BOOKING COUNT	ACTIONS
1	Rahul Sharma	9876543210	rahul.sharma@gmail.com	Male	2	Edit View
2	Priya Reddy	9876543211	priya.reddy@gmail.com	Female	2	Edit View
3	Arjun Kumar	9876543212	arjun.kumar@gmail.com	Male	2	Edit View
4	Sneha Patel	9876543213	sneha.patel@gmail.com	Female	2	Edit View
5	Vikram Rao	9876543214	vikram.rao@gmail.com	Male	2	Edit View
6	Ananya Iyer	9876543215	ananya.iyer@gmail.com	Female	2	Edit View
7	Karthik Reddy	9876543216	karthik.reddy@gmail.com	Male	1	Edit View
8	Meghana Sharma	9876543217	meghana.sharma@gmail.com	Female	1	Edit View

Figure 2. Passenger registry — passenger master data with booking counts and actions.



11. Implementation Details

11.1 Technology Stack

Layer	Technology / Component	Purpose
Frontend	HTML / CSS / JavaScript	Operational UI, forms, navigation and data presentation
Backend	Application server / API	Business logic, validation and database operations
Database	MySQL 8.0	Persistent relational data
Connectivity	MySQL driver / connector	Application-to-database communication
Deployment	Localhost	Local project execution and demonstration

11.2 Database Connectivity

The frontend communicates with the application backend, which performs validated SQL operations against the MySQL database. The UI's 'Database Connected' state shown in the screenshots indicates that the application is designed around a live database-backed workflow rather than static mock data.

11.3 Validation and Integrity

- Primary keys uniquely identify records.
- Foreign keys maintain relationships between master and transactional tables.
- Unique constraints protect phone/email and one-payment/one-cancellation relationships.
- Booking status and payment status are controlled through defined values.
- Seat references are tied to buses and selected through trip bookings.

12. Testing (Test Cases and Results)

Test Case	Expected Result	Status
Database connection	Application connects to MySQL successfully	Verified in UI
Passenger registry	Passenger records load with booking counts	Verified in screenshot
Schedule search	Schedules are filtered by source, destination and date	Implemented
Seat booking	Seat is associated with a booking and passenger	Implemented
Payment tracking	Payment status is associated with booking	Implemented
Cancellation	Cancellation and refund are linked to booking	Implemented
Reports	Aggregated operational metrics are displayed	Verified in screenshot
Referential integrity	Invalid foreign-key references are rejected	Database constraint

13. Conclusion and Future Enhancements

13.1 Conclusion

The Bus Reservation & Fleet Scheduling System provides a normalized relational solution for route, bus, schedule, passenger, booking, crew, payment and cancellation workflows. The connected interface converts the database model into practical operational screens and reporting views. The design demonstrates core DBMS concepts including entity modeling, primary and foreign keys, normalization, joins, aggregation and transaction-oriented data handling.

13.2 Future Enhancements

- Authentication and role-based access control.
- Online payment gateway integration.
- Live seat locking and reservation timeout.
- Automated notifications for booking and cancellation.
- Advanced fleet utilization and revenue analytics.
- Cloud deployment, backups and audit logging.

14. References

- MySQL Documentation.
- Reference DBMS Course Project Report supplied for report structure.
- Project database schema and application implementation.

15. Appendix: Project Repository

GitHub Repository: <https://github.com/samyukthaa21/DBMS-Course-Project>